

## **Report for Differential Privacy:**

### **Introduction:**

The task involves generating synthetic laundering data to balance a dataset containing non-laundering data for machine learning model training. Differential privacy is a technique used to ensure that the synthetic data generated does not reveal sensitive information about individuals in the original dataset. By adding controlled noise to the data, differential privacy helps protect individual privacy while maintaining the overall statistical properties of the dataset. In this task, CTGAN was used to generate synthetic laundering data, with differential privacy techniques applied during training to ensure the privacy of the original data is preserved.

There were approximately 9504852 rows in total. Initially, we chose 10,000 rows of laundering data and 60,000 rows of non-laundering data, which led to an imbalanced dataset. To address this, CTGAN (Conditional Generative Adversarial Network) was used to generate an additional 10,000 rows of laundering data, effectively balancing the dataset. The synthetic data allows for a more balanced dataset, which can be used to train machine learning algorithms more effectively. The dataset was then used to train different algorithms, with Random Forest performing the best. Hyperparameter tuning was applied to further improve model accuracy.

### **Objective:**

- The objective of this task was to:
- Use CTGAN to generate synthetic laundering data and balance it with the non-laundering data.
- Train machine learning models, specifically using the Random Forest algorithm, on the generated dataset.
- Perform hyperparameter tuning to optimize the Random Forest model and improve its accuracy.

### **Methodology:**

#### **Synthetic Data Generation with CTGAN:**

Given the imbalance between the laundering data and non-laundering data, CTGAN was used to generate synthetic data. CTGAN is particularly useful in such situations because it generates realistic, high-dimensional data by learning the conditional relationships between various features in the dataset. The approach used was as follows:

**Initial Dataset:** The dataset initially contained 60,000 rows of non-laundering data and 10,000 rows of laundering data.

**Data Generation:** Using CTGAN, an additional 10,000 rows of laundering data were generated, bringing the total number of laundering data samples to match the non-laundering data.

By training CTGAN on the original dataset, we were able to generate synthetic laundering data that followed the same distribution and statistical properties as the real data, making the dataset more balanced for training machine learning models.

#### **Machine Learning Model – Random Forest:**

After generating the synthetic laundering data, we split the dataset into training and testing sets. Random Forest, an ensemble learning algorithm, was selected as the model for training. Random Forest is particularly effective for imbalanced datasets because it builds multiple decision trees and aggregates their predictions, allowing for better classification of minority classes.

### **Hyperparameter Tuning:**

To improve the accuracy of the Random Forest model, hyperparameter tuning was conducted. Key parameters, such as the number of decision trees (n\_estimators) and the depth of the trees (max\_depth), were optimized using GridSearchCV. This process helps in finding the best combination of parameters for the model, improving its predictive performance on unseen data.

### **TensorFlow for Model Training:**

TensorFlow, an open-source framework developed by Google, was employed to train the CTGAN model. TensorFlow offers several advantages in deep learning tasks, including scalability and GPU acceleration, which are crucial when training complex models like CTGAN.

**Efficient Training:** TensorFlow supports efficient training of deep learning models by utilizing hardware accelerators such as GPUs, significantly reducing training time.

**Deep Learning Support:** It allows for the creation of complex neural network architectures like CTGAN, which require advanced deep learning capabilities to generate high-quality synthetic data.

**Integration with Keras:** TensorFlow integrates well with Keras, a high-level API for building neural networks, enabling rapid prototyping and model tuning.

### **Results and Evaluation:**

#### **Model Performance:**

After training the Random Forest model on the balanced dataset (with both real and synthetic data), the model's performance was evaluated using accuracy, precision, and recall. The Random Forest model performed well, providing accurate classifications between laundering and non-laundering transactions.

#### **Hyperparameter Tuning Results:**

Through hyperparameter tuning, the model's performance was further improved. The optimization of key parameters like n\_estimators and max\_depth helped the model generalize better to unseen data, thereby enhancing its predictive capabilities.

#### **Validation and Testing:**

The synthetic data generated by CTGAN was validated against the real data to ensure that it followed the same distribution and statistical properties. The model was then tested on the balanced dataset, with the final model achieving improved performance metrics compared to a baseline model trained on only the real data.

#### **Key Findings:**

**Effective Use of CTGAN:** CTGAN proved to be an effective tool for generating synthetic data, successfully balancing the dataset by creating realistic laundering data.

**Random Forest's Robustness:** The Random Forest algorithm performed well on the balanced dataset, making it a suitable choice for classification tasks involving imbalanced data.

**Importance of Hyperparameter Tuning:** Hyperparameter tuning played a crucial role in improving the model's accuracy, demonstrating that the model's performance can be optimized by carefully selecting the right parameters.

**Challenges:**

**Synthetic Data Generation:** Ensuring that the synthetic data generated by CTGAN accurately reflected the real data was a challenge. While CTGAN performed well, some fine-tuning of the model was required to generate data that closely mimicked the real dataset.

**Tuning Hyperparameters:** Selecting the optimal hyperparameters for the Random Forest model was time-consuming, but it was necessary for improving accuracy and avoiding overfitting.

**Conclusion:**

This task successfully demonstrated the use of CTGAN for generating synthetic laundering data, addressing the problem of dataset imbalance. The Random Forest model, after hyperparameter tuning, achieved high accuracy in classifying laundering and non-laundering transactions. The combination of synthetic data generation and machine learning model training, coupled with hyperparameter tuning, significantly improved model performance. This methodology can be applied to other domains where data imbalance and privacy are important considerations.